



AI-Powered US Tax & Legal Research System

Hybrid retrieval + Graph RAG over 100 US legal & tax documents —
grounded answers with exact citations.

Live demo ↗ kuldeepmishra3-legal-rag.hf.space

Assignment submission · Kuldeep Mishra

Submission — AI-Powered US Tax & Legal Research System

Live demo: <https://kuldeepmishra3-legal-rag.hf.space> — Ask · Summarize · Explore Citations **Code:** this repository · start with [README.md](#)

A Retrieval-Augmented Generation system for US tax & legal research over **100 curated documents** (Acts, court judgments, legal commentary, IRS tax pubs). Hybrid retrieval (vector + BM25 + RRF) + **Graph RAG** (citation graph) → grounded answers via **Gemini 2.5 Pro** with mandatory citations (document / section / page) and a citation-validation guard.

PRD Final Deliverables → where to find each

#	PRD deliverable	Delivered	Location
1	Architecture diagram (full workflow)	✓	ARCHITECTURE.md + image <code>docs/architecture.png</code> (flowchart + request-lifecycle sequence diagram)
2	Working demo (queries · retrieval · summaries · citations + page refs)	✓	Live: https://kuldeepmishra3-legal-rag.hf.space · Local: FastAPI + web UI (<code>backend/</code> , <code>frontend/</code>)
3	Evaluation report (retrieval, faithfulness, golden set, observations)	✓	<code>evaluation_report.md</code>

PRD Expected Outcome → status

Requirement	Status
Process ~100 US legal & tax documents	✓ 100 docs (30 Acts / 30 Judgments / 30 POV / 10 Tax) → 6,289 chunks
Hybrid Search (Vector + ELK/Elasticsearch keyword)	✓ Qdrant + Elasticsearch BM25 → Reciprocal Rank Fusion
Graph RAG (optional advanced)	✓ Neo4j citation graph (3,203 authorities, 4,075 edges)
Page-level metadata for citations	✓ every answer cites document · section · page
Accurate summaries	✓ <code>/summarize</code> per document
Document names, page numbers, references for verification	✓ clickable citations + source links
Measurable performance on a Golden Set	✓ 100-query golden set → see below
OKF (Open Knowledge Format) normalization	✓ OKF-normalized chunks (<code>chunker.py</code>)

Headline results (100-query golden set)

Metric	Score
Retrieval accuracy (Top-1 / Top-5)	95% / 99%
Faithfulness (RAGAS)	0.96
Answer correctness (LLM judge)	91% correct / 98% correct-or-partial
Grounded / hallucinated	100% / 0%

Full methodology, per-category breakdown, and limitations: [evaluation_report.md](#).

Golden set: `eval/golden_set.csv` — 100 hand-authored `query · ground-truth answer · source document · page` rows.

Run it

Try the live demo (no setup): <https://kuldeepmishra3-legal-rag.hf.space>

Or run locally (full setup in [README.md](#)):

```
python3.10 -m venv venv && ./venv/Scripts/python.exe -m pip install -r requirements.txt &&
./venv/Scripts/python.exe -m pip install -e .
cp .env.example .env # add your Gemini/Vertex credentials
docker compose --profile all up -d # Qdrant + Elasticsearch + Neo4j
./venv/Scripts/python.exe scripts/build_processed.py
./venv/Scripts/python.exe scripts/build_vector_index.py
./venv/Scripts/python.exe scripts/build_keyword_index.py
./venv/Scripts/python.exe -m legalrag.graph.graph_builder
./venv/Scripts/python.exe -m uvicorn backend.app.main:app --port 8000 # → http://localhost:8000
```

Tests: `./venv/Scripts/python.exe -m pytest tests/ -q` — **63 tests** across all phases.

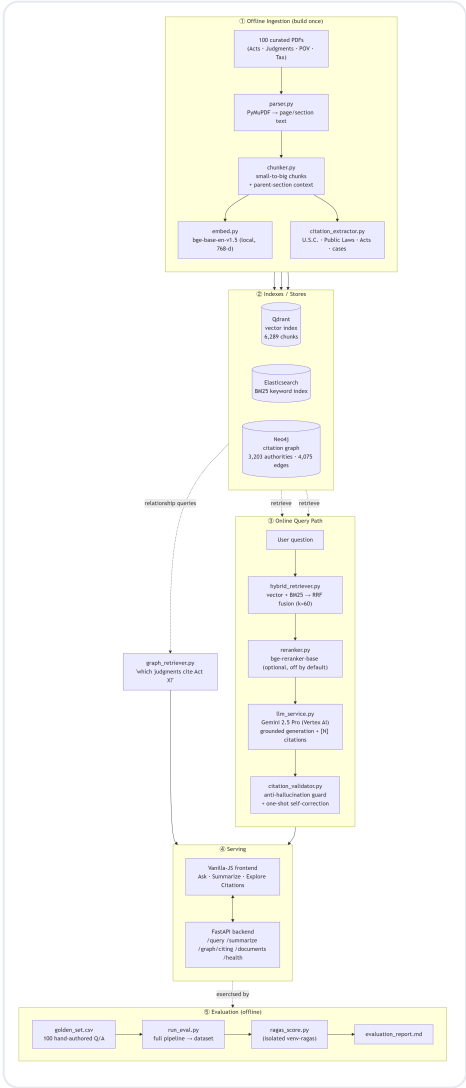
Repo map

- `src/legalrag/` — the pipeline package (ingestion · indexing · retrieval · generation · graph)
- `backend/ · frontend/` — FastAPI API + web UI
- `eval/` — golden set, evaluation harness, RAGAS, correctness judge
- `deploy/huggingface/` — the self-contained Hugging Face Space (server-less variant)
- `docs/` — PRD, plan, task log, architecture image
- `ARCHITECTURE.md · evaluation_report.md · README.md`

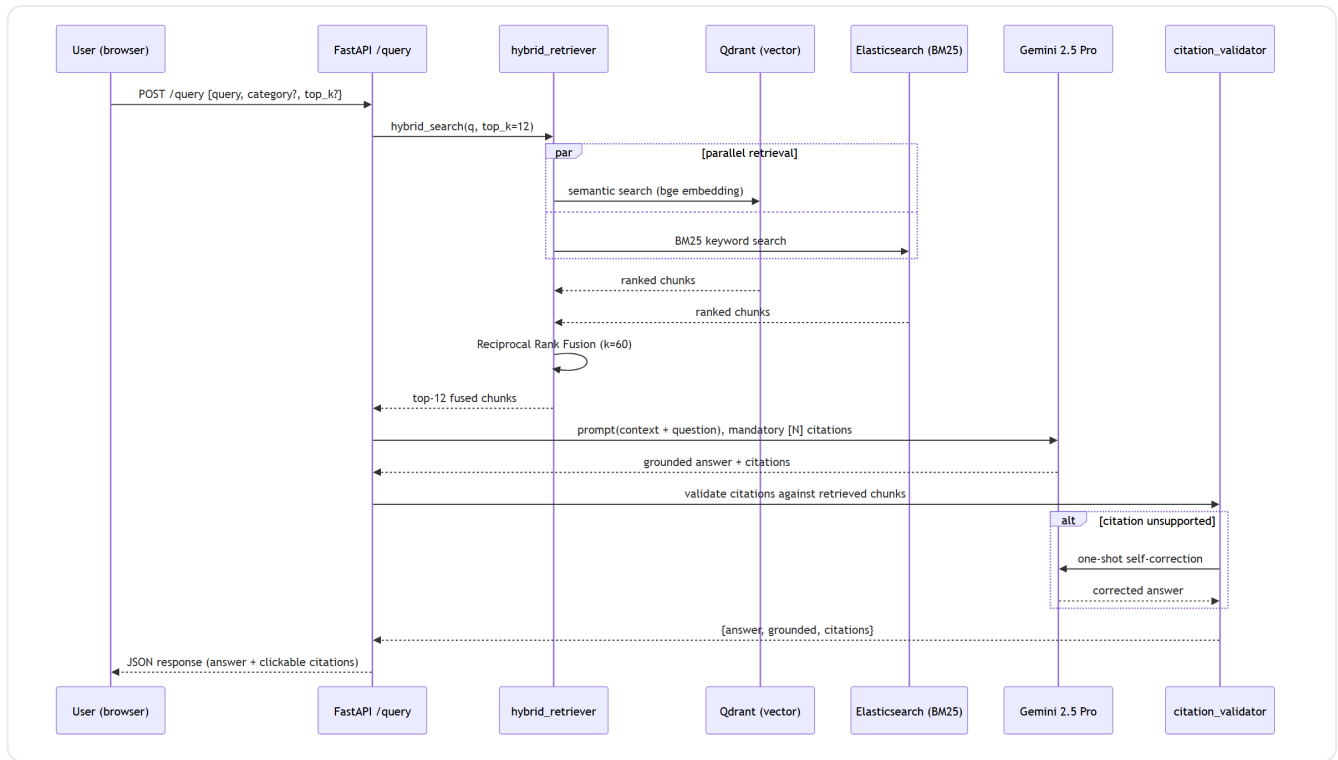
System Architecture — AI-Powered US Tax & Legal Research System

This document describes the end-to-end architecture: how documents become a searchable, citable knowledge base, and how a natural-language question becomes a grounded, cited answer.

1. High-level diagram



2. Request lifecycle (an "Ask" query)



3. Component responsibilities

All pipeline modules live in the `legalrag` package under `src/legalrag/` ; runnable entrypoints live in `scripts/` . Paths below are relative to `src/` .

Layer	Module(s)	Responsibility
Config	<code>legalrag/config.py</code>	Resolves project root + data paths, loads <code>.env</code> (one source of truth)
Parsing	<code>legalrag/ingestion/parser.py</code>	PDF → text with page + section tags (PyMuPDF)
Chunking	<code>legalrag/ingestion/chunker.py</code> , <code>scripts/update_parent_context.py</code>	Small-to-big chunks; each chunk keeps its parent-section text for context
Embedding	<code>legalrag/ingestion/embed.py</code>	Local <code>bge-base-en-v1.5</code> (768-dim), no API cost
Vector index	<code>legalrag/indexing/vector_indexer.py</code> , <code>scripts/build_vector_index.py</code>	Upsert chunk vectors into Qdrant

Layer	Module(s)	Responsibility
Keyword index	<code>legalrag/indexing/es_indexer.py</code> , <code>scripts/build_keyword_index.py</code>	BM25 index in Elasticsearch
Hybrid retrieval	<code>legalrag/retrieval/hybrid_retriever.py</code>	Vector + BM25 → Reciprocal Rank Fusion
Reranking	<code>legalrag/retrieval/reranker.py</code>	<code>bge-reranker-base</code> — off by default (hurt Top-1 on this corpus)
Generation	<code>legalrag/generation/llm_service.py</code>	Gemini 2.5 Pro, grounded answers with mandatory [N] citations
Anti-hallucination	<code>legalrag/generation/citation_validator.py</code>	Validate every citation maps to a retrieved chunk; one-shot self-correction
Graph RAG	<code>legalrag/graph/{citation_extractor,graph_builder,graph_retriever}.py</code>	Neo4j citation graph for relationship queries
API	<code>backend/app/main.py</code>	FastAPI: <code>/query</code> , <code>/summarize</code> , <code>/graph/citing</code> , <code>/documents</code> , <code>/health</code>
UI	<code>frontend/</code>	Vanilla JS/HTML/CSS: Ask · Summarize · Explore Citations
Evaluation	<code>eval/</code>	Golden set, RAGAS (isolated venv), direct correctness judge

4. Key design decisions

- **Hybrid over pure-vector.** Legal text is citation- and term-heavy; BM25 catches exact section numbers and statute names that dense vectors blur. RRF fuses both rankings without tuning weights.
- **Local embeddings (bge-base-en-v1.5).** Removes API rate limits/cost from the ingestion path; `voyage-law-2` was dropped for practical reasons (documented in the evaluation report as a future improvement).
- **Vertex AI for generation.** The new `google-genai` SDK on Vertex authenticates the service-account key and can spend the \$300 GCP credit (AI Studio can't). `thinking_budget=0` disables Gemini-2.5 thinking tokens for predictable output.
- **Citation validation as a guard.** Every `[N]` in an answer must resolve to a retrieved chunk; unsupported citations trigger one-shot self-correction. This is what drives the measured **0% hallucination / 100% grounded** result.
- **Graph RAG for relationships.** "Which judgments cite the Fiscal Responsibility Act?" is a graph traversal, not a similarity search — Neo4j answers it directly.
- **RAGAS quarantined in `venv-ragas`.** Its langchain pins conflict with the main system; decoupling generation (main venv) from scoring (isolated venv) keeps the working system untouched.

5. Data / infra footprint

Store	Docker service	Content
Qdrant	<code>qdrant</code> (profile <code>phase2</code>)	6,289 chunk vectors (768-dim)
Elasticsearch	<code>elasticsearch</code> (profile <code>phase3</code>)	BM25 index over the same chunks
Neo4j	<code>neo4j</code> (profile <code>phase7</code>)	3,203 authority nodes, 4,075 citation edges

Corpus: **100 native-PDF documents** — 30 Acts / 30 Court Judgments / 30 POV / 10 Tax — page band 15–60.

Evaluation Report — AI-Powered US Tax & Legal Research System

Milestone 4: Quality Assurance & Evaluation Evaluated end-to-end on a hand-authored 100-query Golden Set. Answer model: **Gemini 2.5 Pro** (via Vertex AI). Judge model: **Gemini 2.5 Flash** (thinking disabled). Date: 2026-07-11.

1. Method

The complete system was run against all **100 golden-set queries** — hybrid retrieval (vector + keyword, RRF fusion) → grounded generation with mandatory citations → citation validation. Two families of metrics were computed:

- **Deterministic metrics** (computed directly, no LLM judge): retrieval accuracy against the *known* correct source document, and citation-grounding rates from the pipeline's own validator. These are exact and reproducible.
- **RAGAS metrics** (LLM-judged, the industry-standard RAG evaluation framework): faithfulness, context precision/recall, and factual correctness. Run in an isolated environment (see §5) with a custom Gemini-Vertex judge.

The Golden Set: 100 questions, **25 per category** (Acts / Court Judgments / POV / Tax), mixed difficulty (56 easy / 34 medium / 10 hard). Every ground-truth answer was hand-authored and verified against the source PDF text (not from general knowledge).

2. Headline Results

Metric	Score	What it means
Retrieval Accuracy (Top-1)	95.0%	Correct source document ranked #1
Retrieval Accuracy (Top-5)	99.0%	Correct source document in top 5
Faithfulness (RAGAS)	0.96	Answers grounded in retrieved context; ~no hallucination
Context Recall (RAGAS)	0.98	Retrieval captured the evidence needed to answer
Context Precision (RAGAS)	0.74	Retrieved chunks are relevant
Answer Correctness (direct judge)	91% correct (98% correct-or-partial)	Answer conveys the ground-truth fact
Grounded rate	100%	Every answer was cited to sources or a proper refusal
Hallucinated / ungrounded answers	0%	No answer made unsupported claims

3. Retrieval Accuracy (deterministic)

The system finds the correct source document for **95% of queries at rank 1**, and **99% within the top 5**. In 99% of cases the correct document appears in the generation context (top-12 chunks).

	Top-1	Top-5
Acts	100%	100%
POV	100%	100%
Court Judgments	96%	96%
Tax Documents	84%	100%
Overall	95%	99%

By difficulty: easy 91% Top-1, medium 100%, hard 100%.

Observation — Tax is the soft spot (84% Top-1, but 100% Top-5). IRS publications share heavy overlapping vocabulary (deductions, thresholds, filing status), so the *exact* right pub sometimes ranks 2nd–3rd behind a topically-adjacent one. It's always in the top 5, so generation still succeeds. Interestingly, "easy" queries score slightly *lower* at Top-1 (91%) than medium/hard (100%) — easy questions use generic phrasing ("what's the standard deduction?") that matches many documents, whereas harder questions name a specific Act/section that pins retrieval precisely.

4. Faithfulness & Answer Quality (RAGAS)

- **Faithfulness 0.96** — answers are almost entirely supported by the retrieved context. Only **3 of 100** answers scored below 0.5 (potential hallucination). By category: Acts 0.98, POV 0.98, Tax 0.96, Judgments 0.91.
- **Context Recall 0.98** — retrieval reliably surfaced the supporting evidence.
- **Context Precision 0.74** — most retrieved chunks are relevant; the lower figure (Judgments 0.61) reflects court opinions retrieving several same-case chunks, some of which are context rather than the precise answer.
- **Grounding 100% / Refusal 1%** — every answer either cited its sources or correctly refused ("I could not find the answer in the provided legal sources"). One query correctly refused.

Why we did NOT use RAGAS `factual_correctness` — and what we used instead

RAGAS's `factual_correctness` reported **0.23 (F1 mode)** and **0.34 (recall mode)** — both **invalid for this golden set**, and we investigated rather than reporting a misleading number. The metric decomposes answer and reference into atomic *propositions* and runs claim-by-claim NLI. Our golden ground-truth answers are deliberately **terse fragments** — bare figures (`$250,000`) or noun phrases (`An Individual Taxpayer Identification Number (ITIN)`) — which are not propositions, so the NLI step can't "entail" them even when the answer contains them **verbatim**. Concrete false-zeros from the run:

- Reference `$250,000` · answer "...*applies to income over \$250,000 for those Married filing jointly*" → scored **0.0**

- Reference `An Individual Taxpayer Identification Number (ITIN)` · answer *"The IRS will issue an Individual Taxpayer Identification Number (ITIN) to a nonresident..."* → scored **0.0**

Both answers are unambiguously correct. 60 of 100 answers scored exactly 0.0 while being correct — the metric is simply the wrong tool for terse references.

Instead we used a **direct LLM-judge correctness metric** (does the candidate answer convey the reference's key fact? correct / partial / incorrect), which handles terse references correctly:

Verdict	Count
CORRECT	91%
PARTIAL	7%
INCORRECT	2%
Correct-or-partial	98%

By category (CORRECT rate): Acts 100%, Judgments 96%, POV 88%, Tax 80%. This aligns with the deterministic grounding (100%), faithfulness (0.96), and retrieval (95% Top-1): the answers are accurate. The 2 fully-incorrect answers are in the weaker Tax/POV categories (multi-year figure ambiguity and one retrieval miss).

5. RAGAS Integration Notes (methodology transparency)

RAGAS proved non-trivial to run against this stack, and the workarounds are worth documenting:

1. **Dependency isolation.** RAGAS's langchain dependencies conflict with the main system's langchain 1.x (RAGAS hard-imports a module removed in langchain 1.0). Since the core system has zero langchain dependency, RAGAS was quarantined in a separate virtual environment (`venv-ragas` , pinned in `requirements-ragas.txt`) — the working system is completely untouched.
2. **Decoupled generation from scoring.** `eval/run_eval.py` (main venv) runs the real pipeline and dumps `eval/eval_dataset.jsonl` ; `eval/ragas_score.py` (isolated venv) scores it. This is best practice — the expensive generation runs once and can be re-scored freely.
3. **Custom judge.** RAGAS's stock `ChatVertexAI` wrapper failed on this project's Gemini-2.5 models (thinking tokens broke structured-output parsing → all-NaN scores; gemini-2.0-flash isn't served on this project's Vertex). Fixed with a custom `BaseRagasLLM` that calls the same reliable `google-genai` Vertex client the rest of the system uses, with thinking disabled.

6. System Overview (what was evaluated)

- **Corpus:** 100 curated native-PDF documents (30 Acts / 30 Court Judgments / 30 POV / 10 Tax), page band 15–60, parsed into **6,289 page/section-tagged chunks**.
- **Retrieval:** hybrid — vector (bge-base-en-v1.5, local, in Qdrant) + keyword (BM25, Elasticsearch), fused with Reciprocal Rank Fusion.
- **Generation:** Gemini 2.5 Pro over the top-12 retrieved chunks, with mandatory `[N]` citations (doc + section + page) and a citation-validation guard + one-shot self-correction.

- **Graph RAG:** Neo4j citation graph (3,203 authorities, 4,075 citation edges) answering relationship queries pure search cannot ("which judgments cite the Fiscal Responsibility Act?").
 - **Demo:** FastAPI + vanilla-JS web app (Ask / Summarize / Explore-Citations).
-

7. Observations, Limitations, and Improvements

Strengths - Retrieval is strong (95% Top-1, 99% Top-5) and generation is highly faithful (0.96) with 100% grounding — for a legal tool, *not hallucinating* is the critical property, and the system delivers it. - Every answer is traceable to an exact document, section, and page (PRD's core verification requirement). - Graph RAG adds relationship queries that semantic search structurally cannot answer.

Limitations - **Tax retrieval precision (84% Top-1).** Heavy vocabulary overlap between IRS pubs. Mitigated by top-5 = 100%, but Top-1 could improve. - **Multi-year corpus.** Documents span 2025 and 2026 tax years; a year-agnostic question ("standard deduction for single") can surface figures from both years. The answer reports both faithfully but doesn't always label the year on every figure. - **General-purpose embeddings.** We use `bge-base-en-v1.5` (local) rather than a legal-domain-tuned model (voyage-law-2 was dropped for practical rate-limit/cost reasons); a legal-specialized embedder would likely lift Tax Top-1. - **Citation extraction (Graph RAG)** favors precision over recall — some real cross-references are missed (e.g., case names truncated by the extractor).

Suggested improvements (priority order) 1. Add a **tax-year disambiguation** instruction to the generation prompt (attach the year to every dollar figure, reconcile cross-year differences). No re-indexing needed. 2. Try a **legal-domain embedding model** (voyage-law-2 with a paid key, or a fine-tuned bge) to lift Tax Top-1. 3. Add **light chunk overlap or a reranker** *only if* further evaluation shows boundary-straddling misses — current data doesn't justify the cost. 4. Broaden **citation extraction recall** in the graph (fuller case-name capture).

Artifacts

- `eval/golden_set.csv` — the 100-query golden set
- `eval/eval_dataset.jsonl` — full per-query system outputs (question, answer, contexts, ground truth, retrieval rank)
- `eval/ragas_scores.csv` — per-query RAGAS scores
- `eval/run_eval.py` , `eval/ragas_score.py` — the evaluation harness